Transformers for EHR Trajectories

Integrative Post-GWAS Methods

Jiyeon Min
Novo Nordisk Foundation Center for Basic Metabolic Research, University of Copenhagen

UNIVERSITY OF COPENHAGEN

For this session, you will need the 'eir-transformer.ipynb' notebook.

!Runtime note!

- Each **eirtrain** run takes ~30 min in GithubCodespace, so the repository has the exact dataset and checkpoints used to produce every figure. ***TRAIN_FROM_SCRATCH = False*** restores them and everything downstream runs in seconds.

Where we are?

- Two lectures ago: genotypes to 315 trait predictions per person, the GRS vector.
- Last section: EIR itself, the configs and the run folder
- This section: we use those GRS vectors for a different downstream task:
  - Can a GPT-style model generate EHR trajectories? And does adding the GRS vector make it better?

Overview

1. Simulate EHR trajectories for the PennCATH individuals
2. Train a sequence generation transformer on those trajectories with ***eirtrain.***
3. Retrain with GRS as a second, tabular input and compare validation losses.
4. Compare trajectory generation: many samples, averaged, from a bare prompt.
5. Extract and cluster latents to look for groups of individuals with similar overall trajectories.

Why simulate?

- Ideally we would train on real EHR data for genotyped individuals: lab measurements, diagnoses and other events over time.
- We have no health trajectory data for PennCATH, so we simulate instead.
  - Each individual's lab values are drawn from their genetic percentile for that trait, as predicted by the foundation model. A value of 84 means the 84th percentile of genetically predicted LDL
  - We have 315 predicted traits to draw from. Four of them drive the simulation, and 23 different ones become the GRS input later

- **Caveat**: This is an exercise. Both sets of numbers come from the same model. If the GRS input helps, we cannot tell whether that is genetics working or the two prediction sets sharing an origin. **So this is a demonstration of how to combine two modalities in EIR**, not evidence that genetics predicts health trajectories.

Part 1

Sequences and tokens

The training data

- Two columns and 11,200 rows, saved as *reference_data/eir-transformer/df_ehr.csv*: 1,120 individuals with 10 simulated trajectories each. The suffix in the ID says which trajectory.
- It reaches training through *--output_configs*, not *--input_configs*: for sequence generation the input is the same file, and EIR derives that config itself.

ID	Sequence
10002_0	SEX_1 [VISIT_START] AGE_64 LAB_HDL_cholesterol_...
10002_1	SEX_1 [VISIT_START] AGE_61 LAB_HDL_cholesterol_...
10002_2	SEX_1 [VISIT_START] AGE_62 LAB_HDL_cholesterol_...
...	...

One patient is one string

SEX_1

[VISIT_START] AGE_61

LAB_HDL_cholesterol_34%

LAB_Body_mass_index_BMI_IMPED_82%

DX_E11

[VISIT_END] [SEP]

[VISIT_START] AGE_65

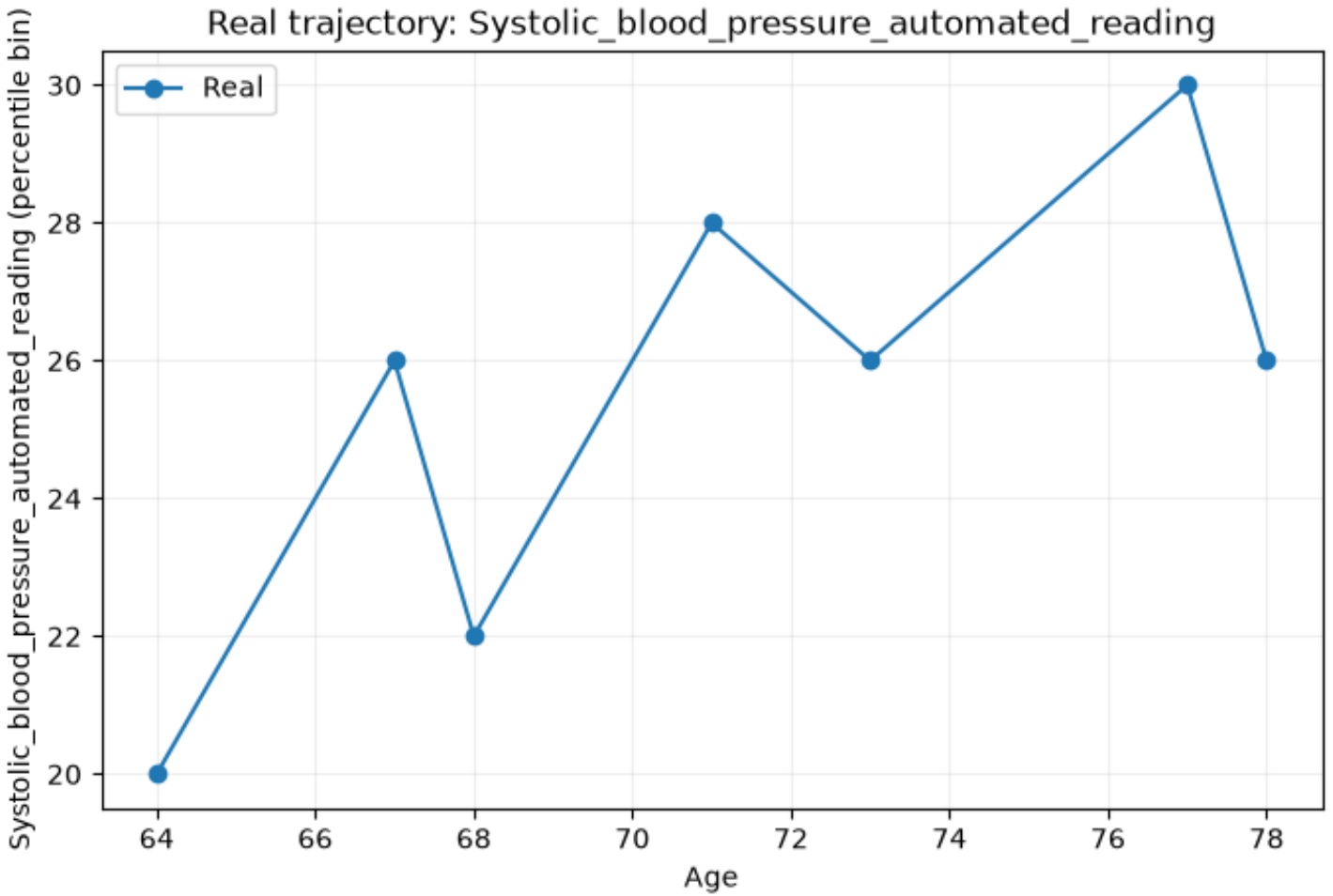
LAB_HDL_cholesterol_36%

LAB_Body_mass_index_BMI_IMPED_88%

DX_E11

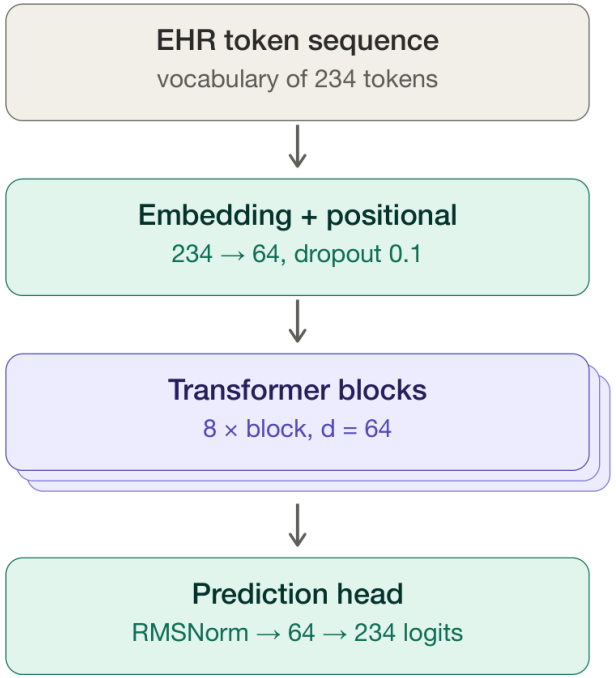
[VISIT_END] ...

A trajectory, decoded back into a curve



Part 2
Training the model

The model



- A decoder-style transformer
  - token embeddings, positional information, a stack of blocks with self-attention, and a head that predicts the next token.
- Trained with the usual next-token objective.
- One thing to know. When only an output configuration is passed for sequence generation, EIR clones it to set up the earlier part of the model.
  - That is why the diagram shows 8 transformer blocks even though we wrote num_layers: 4.

How a token becomes a vector

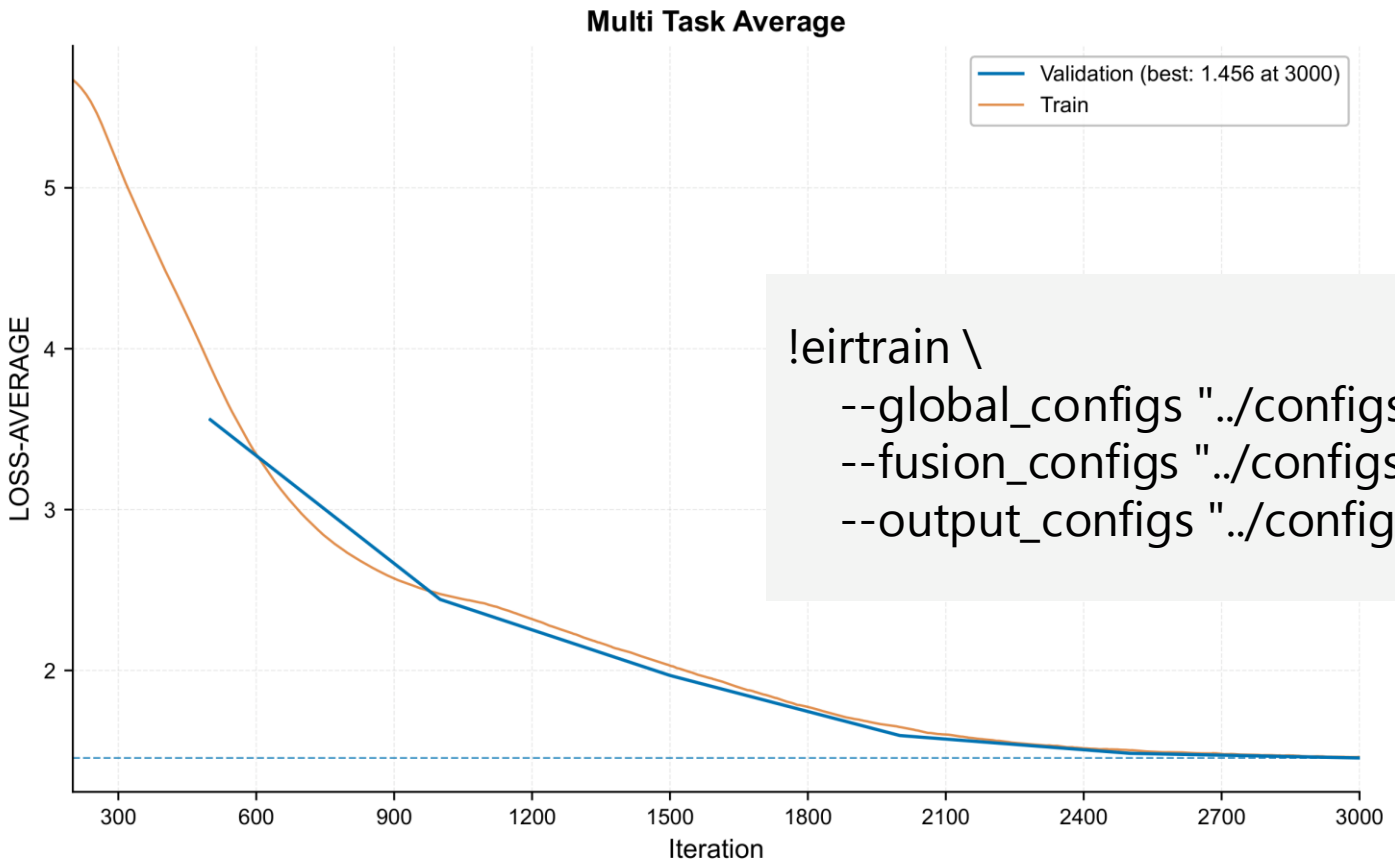
- Every vocabulary entry gets a learned 64-dimensional vector
- Similar tokens end up with similar vectors, if the data supports it
  - LAB_HDL_34% and LAB_HDL_36% should land near each other
- A position vector is added on top, because attention has no sense of order
  - Shuffle the tokens and attention alone cannot tell the difference
  - Which previous tokens does it need to see to guess the next token?
- The token vectors are learned.

Training

Three flags: no input config, but fusion is required (pass-through)

```
!eirtrain \
    --global_configs "../configs/eir-transformer/globals.yaml" \
    --fusion_configs "../configs/eir-transformer/fusion.yaml" \
    --output_configs "../configs/eir-transformer/output_sequence.yaml”
```

Training loss




```
!eirtrain \
    --global_configs "../configs/eir-transformer/globals.yaml" \
    --fusion_configs "../configs/eir-transformer/fusion.yaml" \
    --output_configs "../configs/eir-transformer/output_sequence.yaml"
```

Loss per iteration, plotted automatically into the run folder as training_curve_LOSS-AVERAGE.pdf.

Part 3
Generating sequences

How generation is set up

- Cut a real test patient's history at the first [VISIT_END]
- That prefix becomes the prompt
- Write it into the test config as a manual input
- Run ***eirpredict*** with the trained checkpoint
- The generated sequences are written to results/ehr/ehr/samples/0/manual/

Decoding: how a token gets chosen

- The model scores all 286 vocabulary entries, discards the unlikely ones, and draws one at random in proportion to the scores. Not the top one, which is why the same prompt gives a different answer each time.

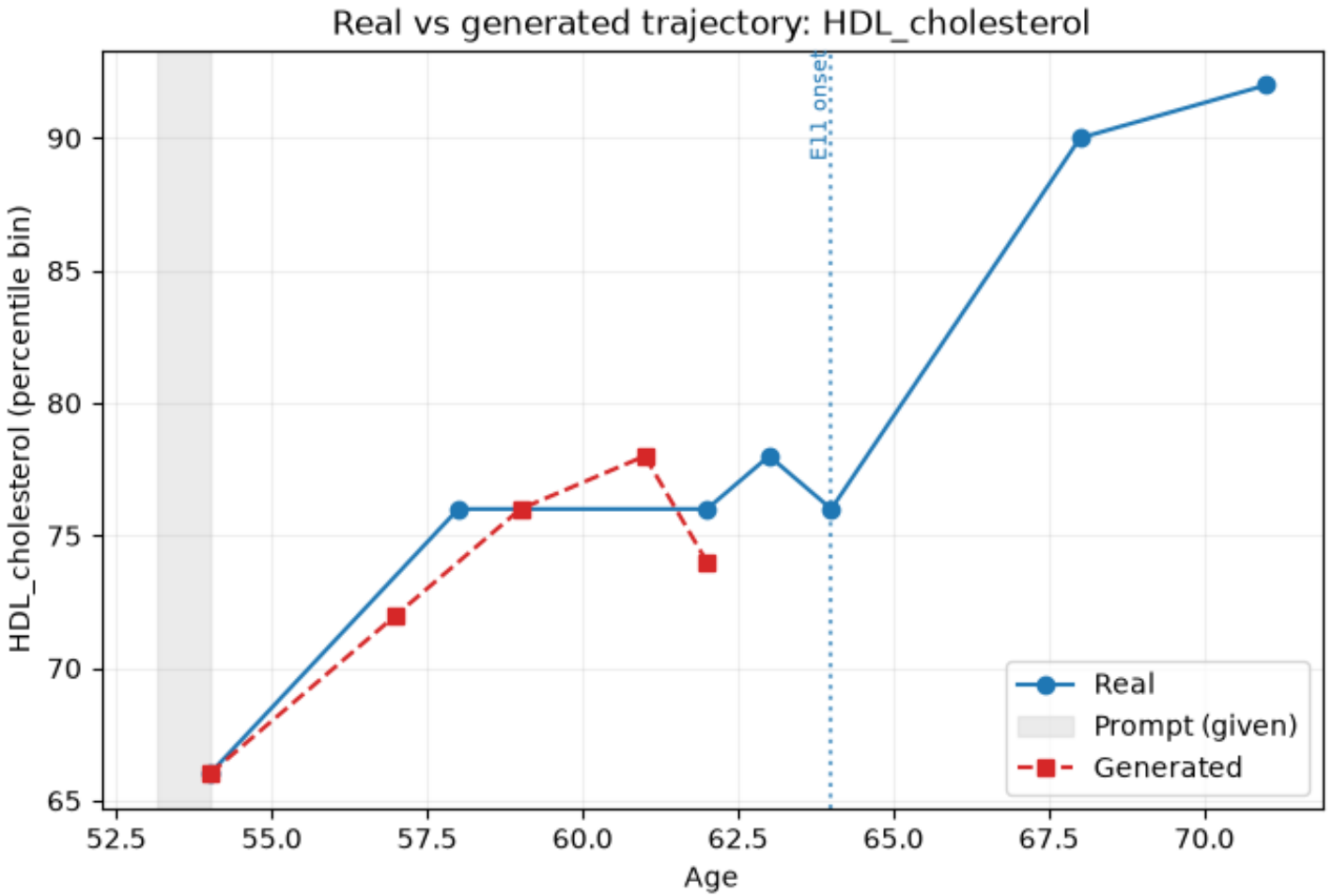
- A handful of sampling settings control how adventurous that draw is. We left them at EIR's defaults.

Three sequences, one individual

- Prompt: 8 tokens (SEX_2 [VISIT_START] AGE_54 LAB_HDL_cholesterol_66% LAB_Body_mass_index_BMI_IMPED_78% LAB_Total_Triglycerides_32% LAB_Systolic_blood_pressure_automated_reading_18% [VISIT_END])
- full_history and generated start with the same 8 tokens, since that is what we handed over. They differ from the second visit onwards, and that is what we compare.

	What it is	Length here
prompt	the first visit only, which is what we give the model	8 tokens
full_history	the record this person actually has in the test set	59 tokens
generated	what the model produces when we ask it to continue the prompt	40 tokens

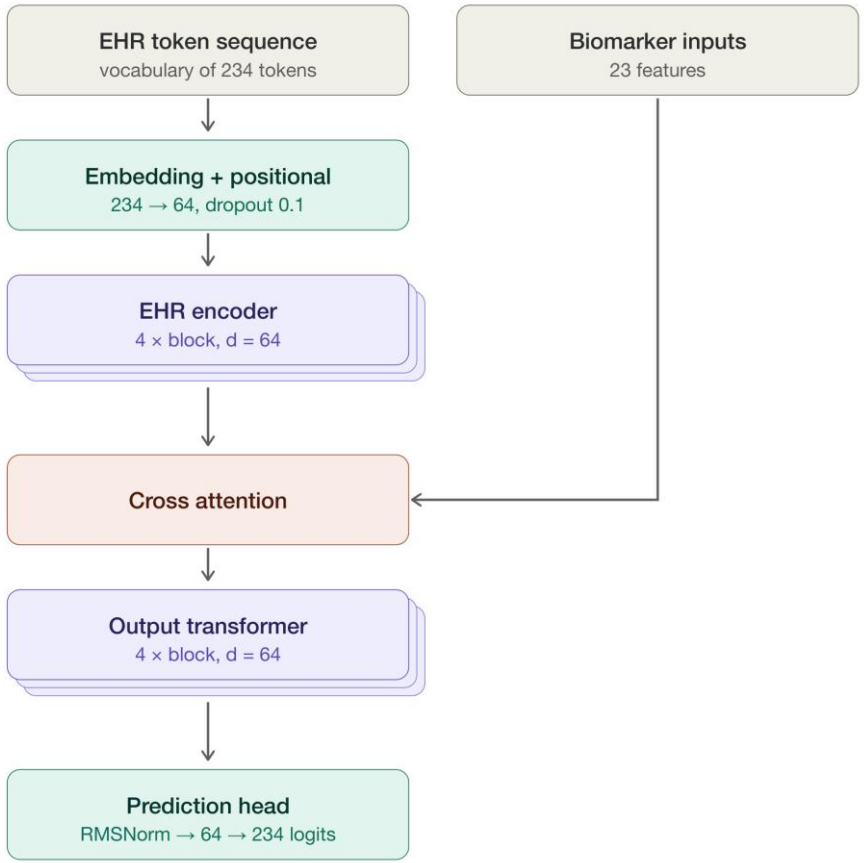
Real (Simulated) against generated



Part 4

Adding GRS as a second input

The model with a second input



- The sequence side is unchanged. A tabular input branch is added, carrying the 23 biochemistry GRS values.
- Hypothesis: knowing this person's genetic biochemistry profile should help the model place their trajectory correctly, especially before any lab values have been observed.

What the GRS model actually receives

Trait	Value	
Cholesterol	5.618	
LDL_direct	3.412	
Glycated_haemoglobin_HbA1c	37.30	
Apolipoprotein_A	1.604	... 23 in total

Preparing the input

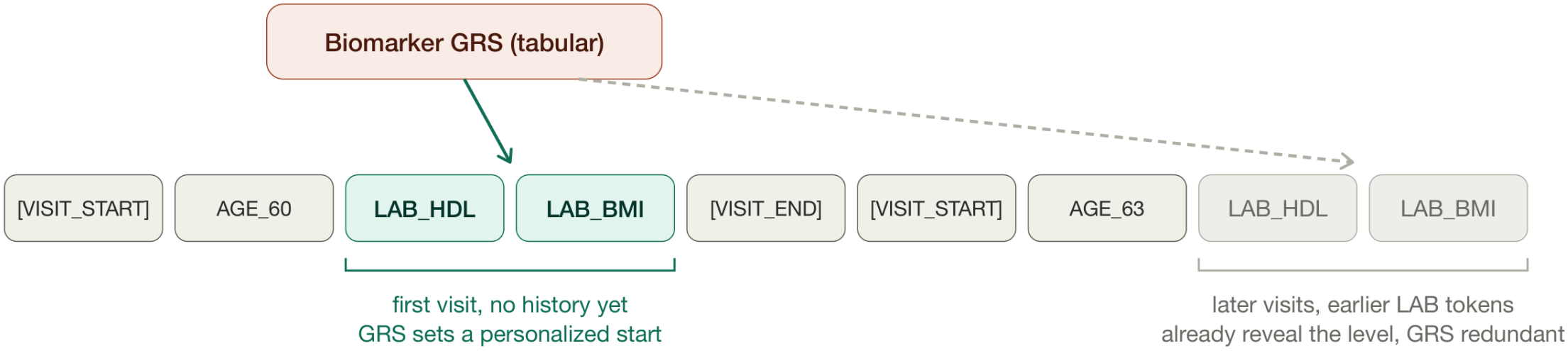
- 23 biochemistry scores per person, one row each
- Listed explicitly under input_con_columns, not 'use everything'
- One extra flag on eirtrain: --input_configs
- The two runs differ by exactly one file
- Same validation IDs, so the losses are comparable

```
!eirtrain \
    --global_configs "../configs/eir-transformer-with-grs/globals.yaml" \
    --input_configs  "../configs/eir-transformer-with-grs/input_tabular.yaml" \
    --fusion_configs "../configs/eir-transformer/fusion.yaml" \
    --output_configs "../configs/eir-transformer-with-grs/output_sequence.yaml"
```

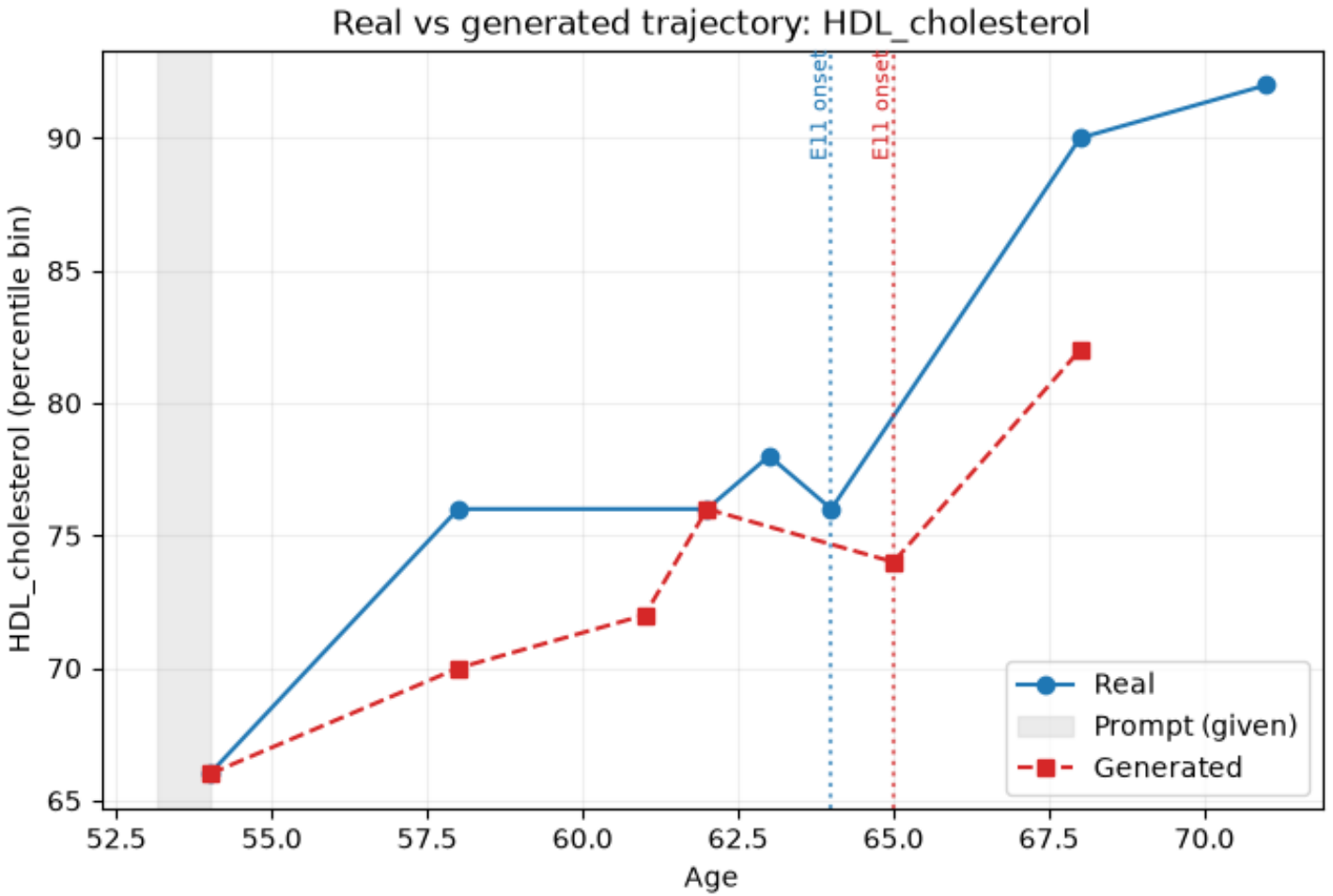
Validation loss: with and without GRS

Why so little difference?

- Lab values are generated from the genetic percentiles
- After a visit or two, the LAB tokens already reveal the person
- So the GRS is redundant for most of the sequence



The same individual, now with GRS



Part 5
Comparing trajectories

Why you cannot judge from one generated sequence

- As with ChatGPT, asking the same question twice gives slightly different answers.
- The model samples from a distribution over the vocabulary at each step.
- An unlucky first token conditions everything after it.
- So draw 64 samples from the same prompt.
- Average them at each visit, both age and values.
- Then a single bad draw cannot dominate.

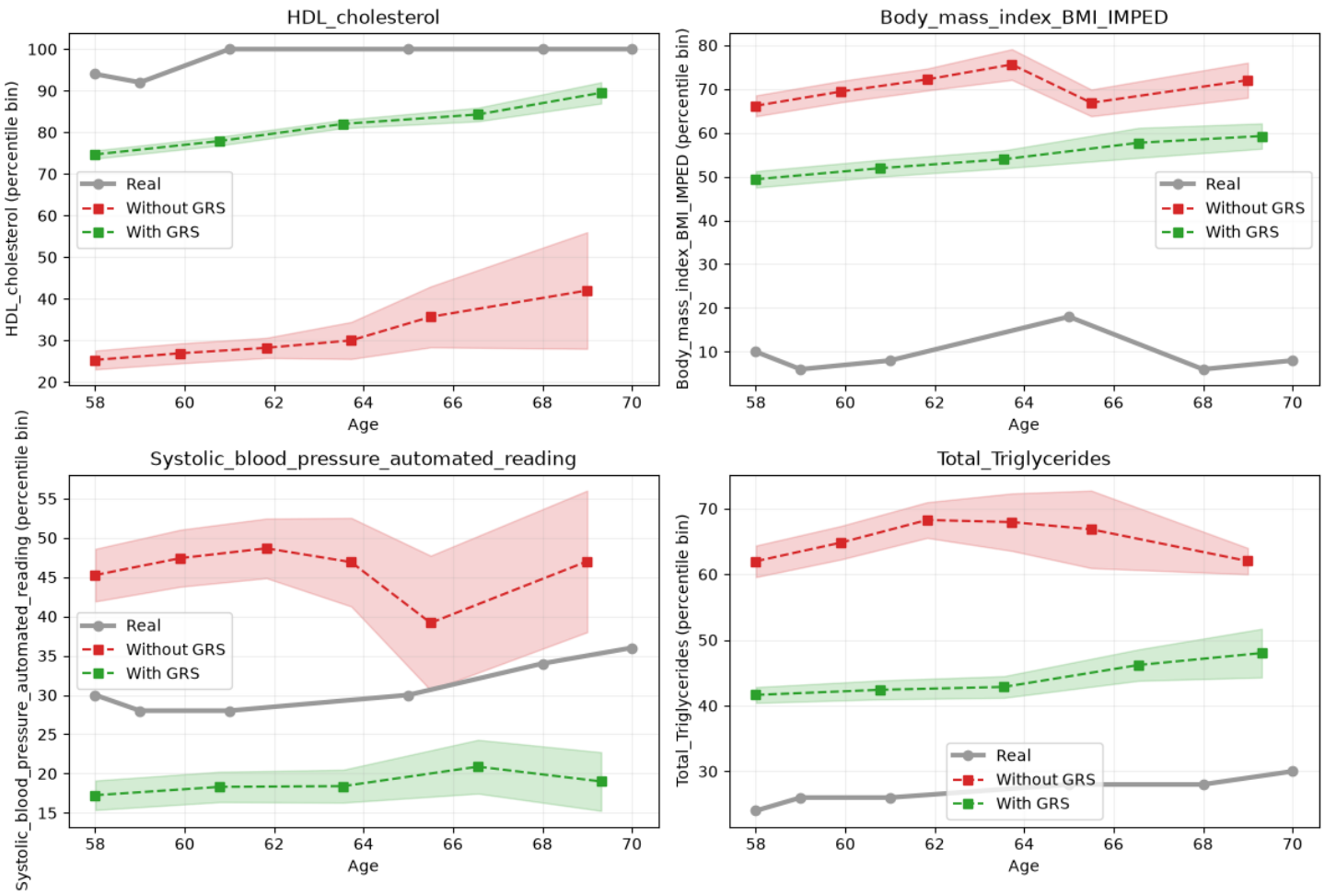
This time the prompt carries no lab values

```
sample_index = 0
full_history = ehr_utils.load_test_sample(index=sample_index)
visit_start_prompt = trim_sequence(full_history, stop_when=lambda t: t.startswith("AGE_"))

print(visit_start_prompt)
  ->  SEX_2 [VISIT_START] AGE_58
```

This time we cut the history much earlier, at the first AGE_ token. Three tokens, and not a single lab value.

Averaged trajectories from the prompt with no lab values



- **Starting point:**
  - Without GRS, the model starts around the population mean for each biomarker.
  - With GRS: The model uses tabular values to start closer to the individual, especially for HDL cholesterol.
- **Trajectory**: Both models drift further from the real trajectory over time.
- **Uncertainty**: Later visits are less reliable as prediction bands widen.

Mean absolute error against the real trajectory

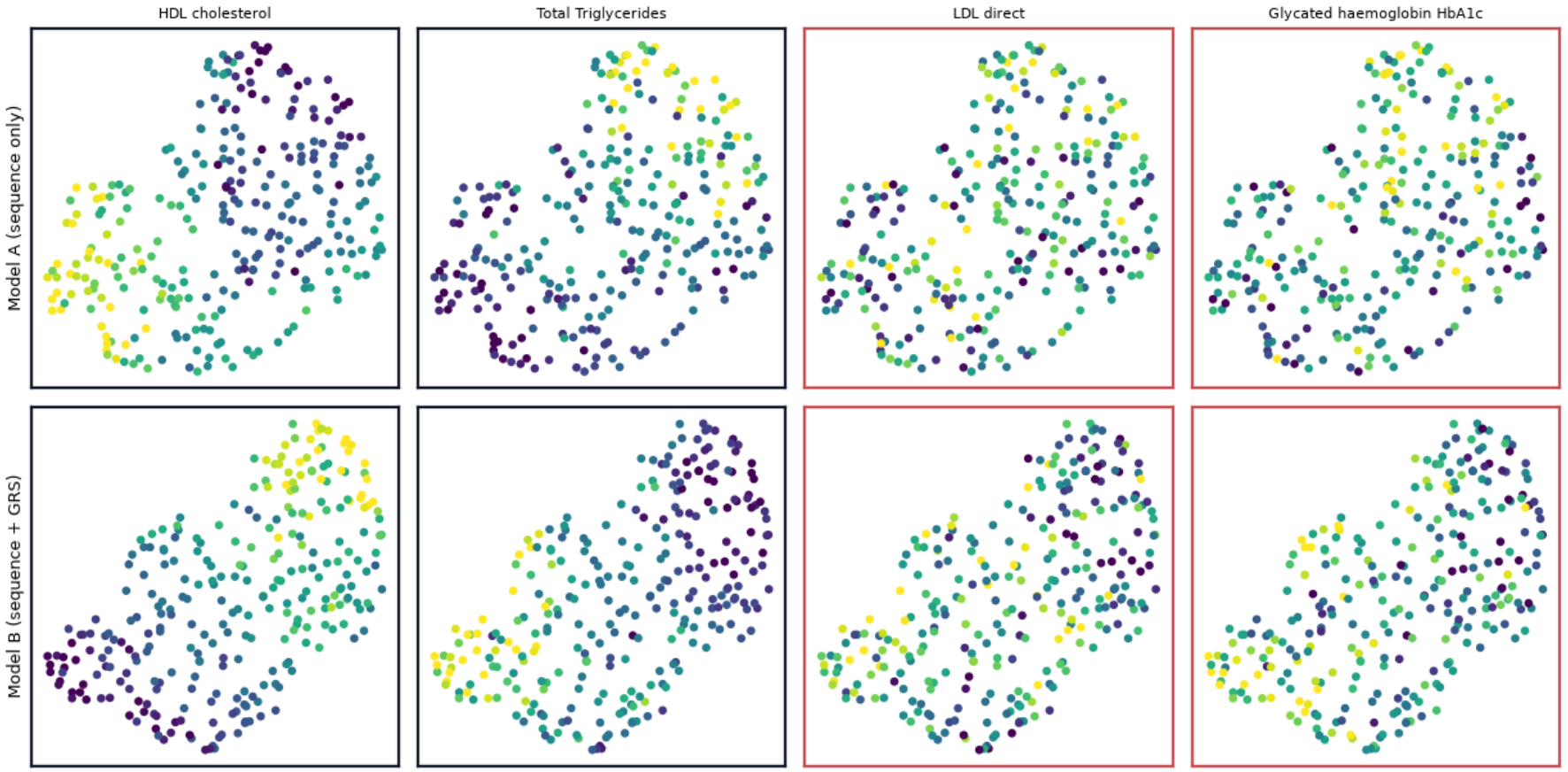
Biomarker	Without GRS	With GRS
HDL cholesterol	66.3	14.3
Systolic blood pressure	14.7	11.2
BMI	61.0	46.5
Triglycerides	38.3	18.5

Part 6
Latent representations

Extracting latents

- Push each test patient's sequence through the trained model
- *latent_sampling* wrote out the *final_norm* layer, the last one before the model predicts a token
- Each sequence gives a 64 × 64 table: one row per token position
- Average down the rows, so every sequence becomes 64 numbers regardless of how long the record was
- Each patient has 10 simulated trajectories, so average those 10 as well
- 2,810 sequences become 281 patients, each 64 numbers, reduced to 2D with UMAP and colored by GRS -> **281 × 2**

Latent space of patient trajectories



Why this matters

- Take everyone in a cohort who develops type 2 diabetes
- Cluster their trajectory embeddings
- Distinct clusters suggest distinct routes to the same diagnosis
  - years of high BMI, or rising HbA1c without obesity
- A single-trait PGS cannot ask this question at all

Summary

- Sequence generation needs no input config, Adding a second modality was one extra config file
- Averaged loss showed no benefit from GRS; the metric was diluted
- Prompted with only sex and age, the GRS model tracks the individual
- Do not judge a generative model from one sample. Draw many, 64 here, and average.
- Latents let you cluster/look at whole trajectories

Questions?